Petit TP Kafka

Introduction à Kafka

Installation

Récupérer l'archive Kafka (si vous consultez ces slides après la formation, pensez à vérifier s'il n'existe pas un mirror link vers une version plus récente) :

```
$ curl https://archive.apache.org/dist/kafka/3.9.0/kafka_2.13-3.9.0.tgz -o
                    <dossier_de_votre_choix>/kafka.tgz
```

Créer un dossier dans lequel extraire cette archive :

```
$ mkdir kafka
$ cd kafka
```

Y extraire l'archive :

```
$ tar -xzf ../<dossier_de_votre_choix>/kafka.tgz
```


Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Nous allons lancer Kafka avec ZooKeeper (KRaft est une alternative).

Rentrer dans le dossier créé et lancer le service ZooKeeper :

```
$ bin/zookeeper-server-start.sh config/zookeeper.properties
```

Ouvrir un nouveau terminal, se rendre dans le même dossier et lancer le broker Kafka :

```
$ bin/kafka-server-start.sh config/server.properties
```


Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Avant d’écrire vos premiers événements, vous devez créer un topic.

Ouvrir un nouveau terminal et exécuter la commande suivante :

```
$ bin/kafka-topics.sh --create --topic vive-la-sg --bootstrap-server localhost:9092
```

Tous les outils en ligne de commande de Kafka disposent d'options supplémentaires : vous pouvez exécuter la commande kafka-topics.sh sans aucun argument pour afficher des informations sur l'utilisation. Par exemple, pour afficher le nombre de partitions du nouveau topic :

```
$ bin/kafka-topics.sh --describe --topic vive-la-sg --bootstrap-server localhost:9092
```


Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Un client Kafka communique avec les brokers Kafka via le réseau pour écrire (ou lire) des événements. Une fois reçus, les brokers stockent les événements de manière durable et tolérante aux pannes aussi longtemps que vous en avez besoin (voire de manière indéfinie).

Exécuter le client console producer pour écrire quelques événements dans votre topic. Par défaut, chaque ligne que vous saisissez entraînera l'écriture d'un événement distinct dans le sujet.

```
$ bin/kafka-console-producer.sh --topic vive-la-sg --bootstrap-server localhost:9092
```

(Clrl-C permet d’arrêter le producer, mais ne le faites pas tout de suite.)

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Ouvrez une autre session de terminal et exécutez le client consommateur de console pour lire les événements que vous venez de créer :

```
$ bin/kafka-console-consumer.sh --topic vive-la-sg --from-beginning --bootstrap-server localhost:9092
```

Vous pouvez arrêter le client consommateur avec Ctrl-C à tout moment, mais ne le faites pas tout de suite.

Revenir au terminal du producer et écrire un nouveau message.

Visualiser comment cet événement apparait dans le terminal du consumer.

Comme les événements sont stockés de manière durable dans Kafka, ils peuvent être lus autant de fois et par autant de consommateurs que vous le souhaitez.

Ouvrir encore un nouveau terminal et réexécuter la commande précédente. Vous pouvez jouer avec différents timings et la présence ou non de l'option --from-beginning.

Ensuite vous pouvez arrêter le consumer et le producer (conserver Zookeeper et le broker, ou relancer ces deux serveurs).

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Kafka Connect permet d'ingérer continuellement des données depuis des systèmes externes vers Kafka, et vice versa. C'est un outil extensible qui exécute des connecteurs, lesquels mettent en œuvre la logique personnalisée pour interagir avec un système externe.

Le fichier connect-file-3.7.0.jar est une bibliothèque JAR qui contient un connecteur prédéfini pour Kafka Connect. Ce connecteur spécifique est utilisé pour lire des données à partir de fichiers locaux et les écrire dans un topic Kafka, ainsi que pour lire des données à partir d'un topic Kafka et les écrire dans des fichiers locaux.

L'ajout de ce fichier JAR au chemin du plugin dans la configuration de Kafka Connect permet à Kafka Connect de trouver et de charger ce connecteur lors de son exécution. Une fois le connecteur chargé, vous pouvez le configurer pour qu'il lise ou écrive des données selon vos besoins spécifiques.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Ouvrir le fichier config/connect-standalone.properties et ajouter la ligne suivante :

```
plugin.path=<chemin_vers_dossier_kafka>/kafka_2.13-3.9.0/libs/connect-file-3.9.0.jar
```

(Adapter le chemin en function de l'OS utilisé.)

Enregistrer le fichier.

Créer un repertoire input quelque part dans votre espace de travail.

Ouvrir le fichier config/connect-file-source.properties et modifier la ligne *file* de la manière suivante :

```
file=<chemin_vers_input>/test.txt
```

Enfin, ouvrir le fichier config/connect-file-sink.properties et modifier la ligne *file* afin de spécifier là où vous souhaitez obtenir la sortie (sink) :

```
file=<chemin_vers_sink>/test.sink.txt
```

Remarque : c'est une bonne pratique de travailler en chemins absolus plutôt que relatifs.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Démarrer deux connecteurs en mode autonome (ils s'exécuteront dans un seul processus local dédié), en fournissant 3 fichiers de configuration en tant que paramètres, de la manière suivante :

```
$ bin/connect-standalone.sh config/connect-standalone.properties
config/connect-file-source.properties config/connect-file-sink.properties
```

Quelques informations sur ces 3 fichiers :

- Le premier est toujours la configuration du processus Kafka Connect, contenant des configurations communes telles que les courtiers Kafka auxquels se connecter et le format de sérialisation des données.
- Les 2 autres fichiers de configuration restants spécifient chacun un connecteur à créer. Ces fichiers comprennent un nom de connecteur unique, la classe de connecteur à instancier, et toute autre configuration requise par le connecteur.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

Ces fichiers de configuration d'exemple, inclus avec Kafka, utilisent la configuration de cluster local par défaut qui a été démarrée précédemment et créent deux connecteurs : le premier est un connecteur source qui lit des lignes à partir d'un fichier d'entrée et les produit dans un topic Kafka, et le deuxième est un connecteur puits qui lit les messages d'un topic Kafka et les écrit chacun dans un fichier de sortie.

Pendant le démarrage, vous verrez plusieurs messages de journalisation, y compris certains indiquant que les connecteurs sont en cours d'instanciation. Une fois que le processus Kafka Connect a démarré, le connecteur source va commencer à essayer de lire les lignes à partir de test.txt et à les produire dans le topic connect-test, et le connecteur puits va commencer à lire les messages à partir du topic connect-test et à les écrire dans le fichier test.sink.txt.

A l’heure actuelle nous n’avons pas créé ce fichier texte. Constater le comportement de Kafka.

Dans un nouveau terminal, aller dans le dossier spécifié et créer un fichier test.txt dans ce dossier, par exemple :

```
echo -e "line1\nline2" > test.txt
```

Vérifier les logs dans le terminal précédent.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Prise en main

On peut vérifier que les données ont été transmises dans l'ensemble du pipeline en examinant le contenu du fichier de sortie :

```
$ more <chemin_vers_sink>/test.sink.txt
```

A noter que les données sont stockées dans le topic Kafka connect-test, donc nous pouvons également exécuter un consommateur de console pour voir les données dans le topic (ou utiliser du code de consommateur personnalisé pour les traiter) :

```
$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic connect-test --from-beginning
```

Les connecteurs continuent de traiter les données. Ajouter des données au fichier et visualiser qu'elles traversent le pipeline :

```
$ echo Another line >> test.txt
```

Vous devriez voir la ligne apparaître dans la sortie du consommateur de console et dans le fichier de destination.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Kafka Streams

A l'adresse suivante se situe une implémentation classique du wordCount en java, adaptée pour Kafka Streams :

https://github.com/apache/kafka/blob/trunk/streams/examples/src/main/java/org/apache/kafka/streams/examples/wordcount/WordCountDemo.java

Si besoin, relancer les commandes suivantes dans des terminaux séparés :

```
$ bin/zookeeper-server-start.sh config/zookeeper.properties

$ bin/kafka-server-start.sh config/server.properties
```


Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Kafka Streams

Créer le topic d'entrée streams-plaintext-input :

```
$ bin/kafka-topics.sh --create \
    --bootstrap-server localhost:9092 \
    --replication-factor 1 \
    --partitions 1 \
    --topic streams-plaintext-input
```

Créer le topic de sortie streams-wordcount-output avec compaction activée (nous reviendrons sur la raison) :

```
$ bin/kafka-topics.sh --create \
    --bootstrap-server localhost:9092 \
    --replication-factor 1 \
    --partitions 1 \
    --topic streams-wordcount-output \
    --config cleanup.policy=compact
```


Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Kafka Streams

Démarrer l'application wordCount :

```
$ bin/kafka-run-class.sh org.apache.kafka.streams.examples.wordcount.WordCountDemo
```

L'application lira à partir du topic d'entrée streams-plaintext-input, effectuera les calculs de l'algorithme WordCount sur chacun des messages lus, et écrira continuellement ses résultats actuels dans le topic de sortie streams-wordcount-output. Par conséquent, il n'y aura aucune sortie STDOUT sauf les entrées de journal au fur et à mesure que les résultats sont écrits dans Kafka.

Démarrer la console du producer dans un terminal séparé pour écrire des données d'entrée dans ce topic :

```
$ bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic streams-plaintext-input
```


Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Kafka Streams

Inspecter la sortie de l'application de démonstration WordCount en lisant à partir de son topic de sortie avec la console du consumer dans un terminal séparé :

```
$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
    --topic streams-wordcount-output \
    --from-beginning \
    --formatter kafka.tools.DefaultMessageFormatter \
    --property print.key=true \
    --property print.value=true \
    --property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
    --property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer
```

Dans l'output, la première colonne est la clé du message Kafka au format java.lang.String et représente un mot qui est compté, et la deuxième colonne est la valeur du message au format java.lang.Long, représentant le dernier décompte du mot.

Continuer à écrire avec la console du producer dans le topic d'entrée streams-plaintext-input.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Kafka Streams

Les sorties de l'application Wordcount sont en fait un flux continu de mises à jour, où chaque enregistrement de sortie (c'est-à-dire chaque ligne dans la sortie originale ci-dessus) est un décompte mis à jour d'un seul mot, également appelé clé d'enregistrement (comme expliqué dans la présentation de Kafka).

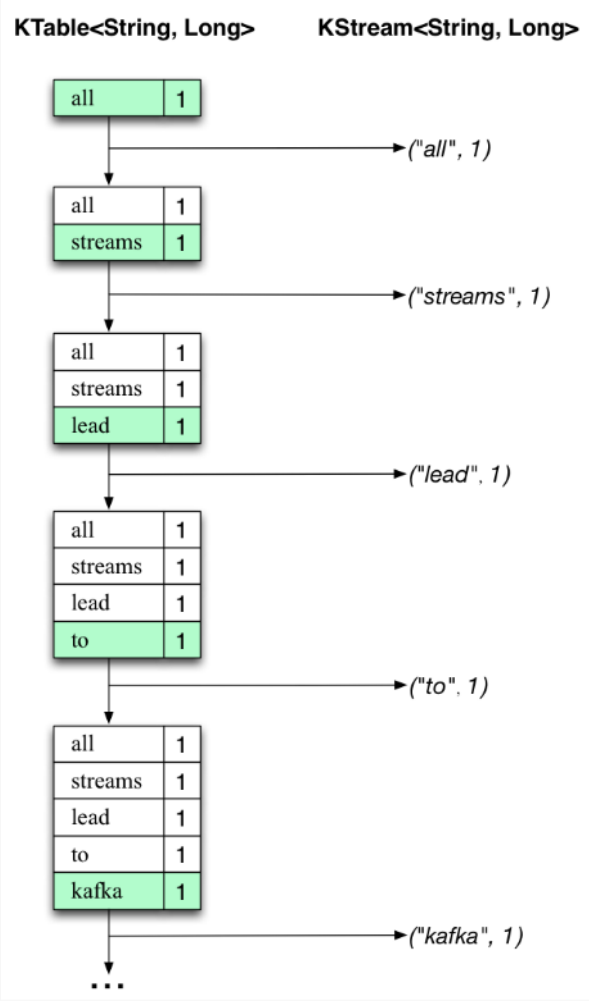
Pour plusieurs enregistrements avec la même clé, chaque enregistrement ultérieur est une mise à jour du précédent.

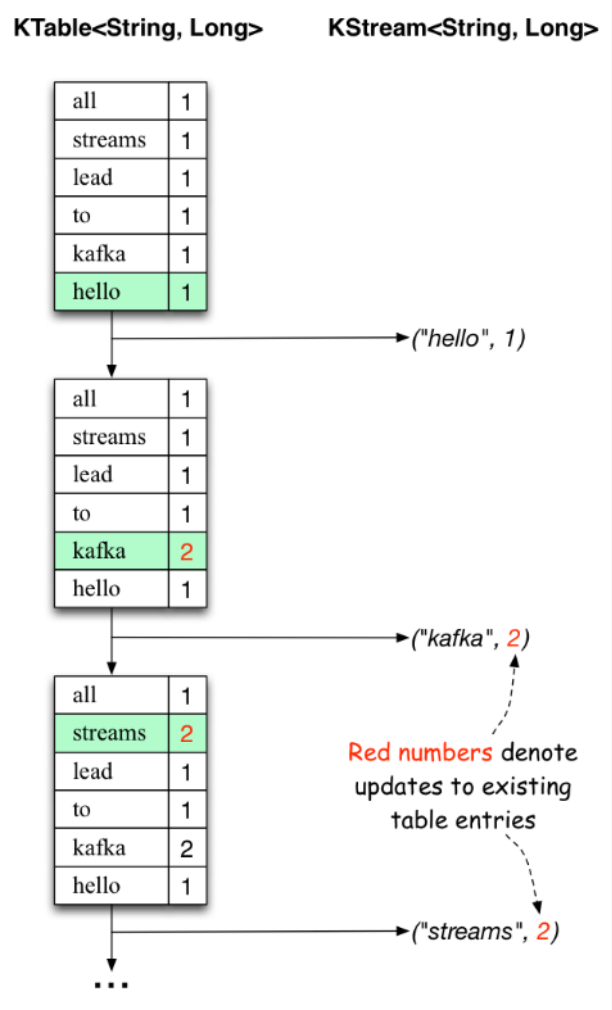
Arrêter le consumer, le producer, l’application Kafka, le broker et le serveur ZooKeeper (CTRL-C).

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.

Introduction à Kafka

Kafka Streams





La première colonne montre l'évolution de l'état actuel de la KTable<String, Long> qui compte les occurrences de mots pour le compteur.

La deuxième colonne montre les enregistrements de changement qui résultent des mises à jour d'état de la KTable et qui sont envoyés au topic Kafka de sortie streams-wordcount-output.

Propriété de B.S. Reproduction complète ou partielle sans autorisation préalable interdite.